

Centre for Development of Advanced Computing (CDAC)

Post Graduate Certificate Program in Advanced Computing (PG-CP AC)

PROJECT REPORT

RailSathi

AI-Powered Railway Reservation System

Submitted By

1. Ayush Phapale
2. Komal Chavan
3. Mitali Raut
4. Pratik Chavan
5. Pratik Dhopte

PG Certificate Program in Advanced Computing (PG-CP AC)

Academic Year 2025–2026

Certificate

This is to certify that the project entitled

"RailSathi — AI-Powered Railway Reservation System"

has been successfully completed by the following students of
Post Graduate Certificate Program in Advanced Computing (PG-CP AC)
at the Centre for Development of Advanced Computing (CDAC).

1. Ayush Phapale
2. Komal Chavan
3. Mitali Raut
4. Pratik Chavan
5. Pratik Dhopte

Project Guide

Head of Department

Date: _____

Acknowledgement

We would like to express our sincere gratitude to the **Centre for Development of Advanced Computing (CDAC)** for providing us the opportunity and infrastructure to undertake this project as part of the PG-CP AC program.

We extend our heartfelt thanks to our **project guide** and faculty members for their constant guidance, valuable suggestions, and encouragement throughout the development of this project.

We are also grateful to our peers and colleagues whose constructive feedback helped us refine the system at various stages of development.

Finally, we thank our families for their unwavering support and motivation during the entire course of this project.

— Ayush Phapale

— Komal Chavan

— Mitali Raut

— Pratik Chavan

— Pratik Dhopte

Abstract

RailSathi is a full-stack, AI-powered railway reservation system designed and developed as the capstone project for the CDAC PG-CP AC program. The system replicates core functionalities of India's railway booking infrastructure — including train search with intermediate stations, multi-class seat management, dynamic fare calculation based on distance, real-time seat availability, waitlist management with automatic promotion, Razorpay-integrated payment processing, and email notifications — while augmenting it with a **Generative AI chatbot** powered by the Groq LLM (LLaMA 3.3 70B) for natural-language train enquiries.

The architecture follows a **microservices-oriented** pattern with four independently deployable components: a **React 18** frontend (Vite), a **Spring Boot 3.5** REST backend, a **FastAPI** GenAI service, and a **Node.js Express** API gateway that unifies routing and cross-origin concerns. Data persistence is handled by **MySQL 8** via Spring Data JPA with Hibernate ORM. Security is implemented through **JWT-based stateless authentication** with BCrypt password hashing and role-based access control (RBAC).

Key engineering highlights include **pessimistic database locking** to prevent seat overselling under concurrent bookings, **HMAC-SHA256 payment signature verification**, AOP-based cross-cutting logging, and a comprehensive DTO layer to decouple the API contract from the persistence model.

Keywords: Railway Reservation, Full-Stack, Spring Boot, React, GenAI, LLM, Razorpay, JWT, Microservices, CDAC

Table of Contents

Chapter	Page
1. Introduction	—
2. Problem Statement & Objectives	—
3. Literature Survey	—
4. System Requirements	—
4.1 Hardware Requirements	—
4.2 Software Requirements	—
5. Software Requirements Specification (SRS)	—
5.1 Functional Requirements	—
5.2 Non-Functional Requirements	—
6. System Architecture	—
7. UML Diagrams	—
7.1 Use Case Diagram	—
7.2 Class Diagram	—
7.3 Sequence Diagrams	—
7.4 Activity Diagrams	—
7.5 State Diagram	—
8. Database Design (ER Diagram)	—
9. Module Description	—
10. User Flow	—
11. REST API Specification	—
12. Implementation Details	—
13. Testing	—
14. Future Enhancements	—
15. Conclusion	—
16. References	—
17. Team Contribution	—

Chapter 1: Introduction

1.1 Background

The Indian Railways is the fourth-largest railway network in the world, carrying over 8 billion passengers annually. The existing online reservation system (IRCTC) handles millions of concurrent transactions, making railway ticketing one of the most demanding real-time systems in the country. Understanding the engineering behind such a system — from concurrent seat management to secure payment processing — provides an excellent learning opportunity for advanced computing students.

1.2 Motivation

The motivation for **RailSathi** stems from the desire to build a production-grade, full-stack application that covers the core CDAC PG-CP AC syllabus topics (Spring Boot, React, MySQL, Spring Security, AOP, Redux) within a single, cohesive project rather than disconnected lab exercises. By choosing the railway reservation domain, the team addresses real-world challenges such as:

- Concurrent access and data integrity (multiple users booking the last seat simultaneously)
- Multi-entity relational data modelling (trains → classes → stops → bookings → passengers)
- Secure, stateless authentication across a decoupled frontend and backend
- Third-party integration (Razorpay payments, Gmail SMTP, Groq LLM)

1.3 Scope

RailSathi is designed as a **web-based railway reservation platform** offering:

- User registration and login with JWT-based security
- Train search with intermediate-station routing and day-of-week filtering
- Multi-class, distance-proportional fare booking for up to 6 passengers per ticket
- Waitlist management with automatic seat promotion on cancellation
- Razorpay test-mode online payment with HMAC signature verification
- Full and partial cancellation with automated refunds
- Email notifications (booking confirmation and cancellation)
- AI-powered chatbot for train enquiries using Groq LLM
- Admin dashboard for train management, booking oversight, and admin creation

1.4 Project Overview

The system follows a **four-service architecture**:

1. **Frontend** — React 18 SPA served by Vite dev server (port 5173)
2. **API Gateway** — Node.js Express reverse proxy (port 8080) routing `/api/genai/*` to the GenAI service and `/api/*` to Spring Boot
3. **Backend** — Spring Boot 3.5 REST API (port 8081) with Spring Data JPA + MySQL

4. **GenAI Service** — Python FastAPI (port 8083) integrating Groq's LLaMA 3.3 70B model via LangChain

Chapter 2: Problem Statement & Objectives

2.1 Problem Statement

Design and implement a full-stack, AI-enhanced railway reservation system that allows passengers to search trains, book and cancel tickets with real-time seat availability, process online payments securely, receive email notifications, and interact with an intelligent chatbot — while administrators manage trains and monitor all bookings through a dedicated dashboard.

2.2 Objectives

1. To build a **secure, role-based** web application with JWT authentication and BCrypt password hashing.
2. To implement **concurrency-safe seat booking** using pessimistic database locking to prevent double-selling.
3. To support **intermediate-station routing** and distance-based proportional fare calculation.
4. To integrate **Razorpay payment gateway** (test mode) with server-side HMAC-SHA256 signature verification.
5. To provide **waitlist management** with automatic promotion when seats become available.
6. To offer **full and partial ticket cancellation** with automated refund processing.
7. To send **email notifications** on booking confirmation, cancellation, and waitlist promotion.
8. To integrate a **Generative AI chatbot** using Groq LLM for natural-language train enquiries.
9. To implement the **complete CDAC syllabus topics** (Spring Boot, React, Redux, JPA, AOP, Security) in a cohesive application.
10. To maintain **clean architecture** with proper layering (Controller → Service → Repository), DTOs, and separation of concerns.

Chapter 3: Literature Survey

3.1 Existing Systems

System	Technology	Key Features	Limitations
IRCTC	Java/J2EE, Oracle DB	Complete reservation, Tatkal, tourism, meals	Proprietary, complex UI, no AI assistant
RedBus	Microservices, AWS	Bus booking, user reviews, GPS tracking	Bus-only, no rail integration
Trainman	Mobile-first	PNR status, train info, seat availability	Read-only, no booking capability
RailYatri	Mobile App	Live train tracking, crowd prediction	Limited booking, ad-supported

3.2 Technology Comparison

Aspect	Existing Approaches	Our Approach (RailSathi)
Backend	Monolithic J2EE / PHP	Spring Boot 3.5 REST + Microservice GenAI
Frontend	Server-side JSP / jQuery	React 18 SPA with Redux state management
Authentication	Session-based (cookies)	Stateless JWT with BCrypt hashing
Concurrency	Application-level locks	Database-level pessimistic locking (SELECT FOR UPDATE)
AI Integration	Rule-based FAQ bots	LLM-powered chatbot (Groq LLaMA 3.3 70B via LangChain)
Payment	Custom payment forms	Razorpay SDK with HMAC signature verification

3.3 Key Takeaways

The literature survey reveals that most existing railway booking systems are either proprietary monoliths (IRCTC) or read-only information apps (Trainman, RailYatri). RailSathi differentiates itself by combining a modern, decoupled architecture (React + Spring Boot + FastAPI) with an AI-powered assistant and a focus on demonstrating concurrency safety and payment security — making it ideal as both a learning project and a functional prototype.

Chapter 4: System Requirements

4.1 Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3 (Dual Core)	Intel Core i5 / AMD Ryzen 5 (Quad Core)
RAM	4 GB	8 GB or higher
Hard Disk	20 GB free space	50 GB SSD
Display	1366 × 768 resolution	1920 × 1080 Full HD
Network	Broadband Internet (for API keys, npm packages)	Broadband Internet

4.2 Software Requirements

Category	Software	Version	Purpose
Operating System	Windows 10/11, macOS, Linux	Latest	Development & deployment
JDK	Oracle JDK / OpenJDK	21	Spring Boot backend compilation & runtime
Build Tool	Apache Maven	3.9+	Java dependency management & build
Node.js	Node.js	18+ LTS	Frontend (Vite) & API Gateway (Express)
npm	Node Package Manager	9+	JavaScript package management
Python	Python	3.10+	GenAI microservice (FastAPI)
Database	MySQL	8.0 / 9.0	Relational data persistence
IDE (Backend)	IntelliJ IDEA / Eclipse STS	Latest	Java development
IDE (Frontend)	VS Code	Latest	React development
Browser	Google Chrome / Firefox	Latest	Application testing
Version Control	Git + GitHub	Latest	Source code management
API Testing	Postman	Latest	REST API testing

4.3 Third-Party Services

Service	Provider	Purpose
Payment Gateway	Razorpay (Test Mode)	Online payment processing, refunds

Email SMTP	Gmail (App Password)	Booking confirmation & cancellation emails
LLM API	Groq Cloud (LLaMA 3.3 70B)	AI chatbot responses

Chapter 5: Software Requirements Specification (SRS)

5.1 Functional Requirements

FR-01: User Registration & Authentication

ID	Requirement	Priority
FR-01.1	Users shall register with name, email, and password	High
FR-01.2	Passwords shall be stored as BCrypt hashes, never in plaintext	High
FR-01.3	Login shall return a signed JWT token (HS256, 24h expiry)	High
FR-01.4	Users shall be able to reset forgotten passwords via emailed single-use tokens (30-min expiry)	Medium
FR-01.5	Users shall be able to update their profile (name, phone) and change their password	Medium

FR-02: Train Search

ID	Requirement	Priority
FR-02.1	Users shall search trains by source station, destination station, and journey date	High
FR-02.2	Search shall support intermediate stations (any two stops on the route in travel order)	High
FR-02.3	Search results shall filter by the day of the week the train runs	High
FR-02.4	Each result shall show available classes with seat availability, full fare, and part-journey fare	High

FR-03: Booking & Payment

ID	Requirement	Priority
FR-03.1	Logged-in users shall book tickets for 1–6 passengers (name, age, gender each)	High
FR-03.2	Fare shall be calculated proportionally by distance for part-journey bookings	High
FR-03.3	Each confirmed passenger shall receive a unique seat number per train + class + date	High
FR-03.4	When Razorpay is enabled, bookings shall stay PENDING until payment is verified via HMAC signature	High
FR-03.5	A unique 10-digit PNR shall be generated for each booking	Medium

FR-04: Waitlist Management

ID	Requirement	Priority
FR-04.1	When a class is full, users may opt to join the waitlist (WL1, WL2, ...)	High
FR-04.2	When a seat is freed via cancellation, the earliest waitlisted passenger shall be automatically promoted to CONFIRMED with a seat number	High
FR-04.3	Promoted passengers shall receive an email notification	Medium

FR-05: Cancellation & Refund

ID	Requirement	Priority
FR-05.1	Users shall cancel an entire booking (all passengers)	High
FR-05.2	Users shall cancel individual passengers (partial cancellation)	High
FR-05.3	Cancellation of a paid booking shall trigger a Razorpay refund for the cancelled seat(s)	High
FR-05.4	Booking status shall be derived from passenger statuses (CONFIRMED, PARTIALLY_CANCELLED, CANCELLED)	High

FR-06: Admin Functions

ID	Requirement	Priority
FR-06.1	Admins shall add, edit, and delete trains (with classes and route stops)	High
FR-06.2	Admins shall view all bookings in the system	High
FR-06.3	Admins shall create additional admin accounts	Medium

FR-07: AI Chatbot

ID	Requirement	Priority
FR-07.1	A floating chatbot widget shall be available on all pages	Medium
FR-07.2	The chatbot shall fetch live train data from the database to answer queries	Medium
FR-07.3	The chatbot shall handle both railway-specific and general knowledge questions	Low

5.2 Non-Functional Requirements

ID	Category	Requirement
----	----------	-------------

NFR-01	Security	All passwords shall be BCrypt-hashed. JWT tokens signed with HS256. CSRF disabled (stateless API). Payment signatures verified server-side.
NFR-02	Performance	Seat availability shall be calculated in real-time via database query (not cached counters) to ensure consistency.
NFR-03	Concurrency	Simultaneous bookings for the same train shall be serialised via pessimistic row locking to prevent overselling.
NFR-04	Reliability	Payment failures and email failures shall not corrupt booking state (transactional rollback).
NFR-05	Usability	The UI shall be responsive and accessible on desktop browsers (Chrome, Firefox, Edge).
NFR-06	Maintainability	Clean layered architecture: Controller → Service → Repository. DTOs decouple API from persistence.
NFR-07	Scalability	Stateless backend enables horizontal scaling. GenAI service is independently scalable.
NFR-08	Logging	AOP-based cross-cutting logging on all controller methods with execution time tracking.

Chapter 6: System Architecture

6.1 High-Level Architecture Diagram

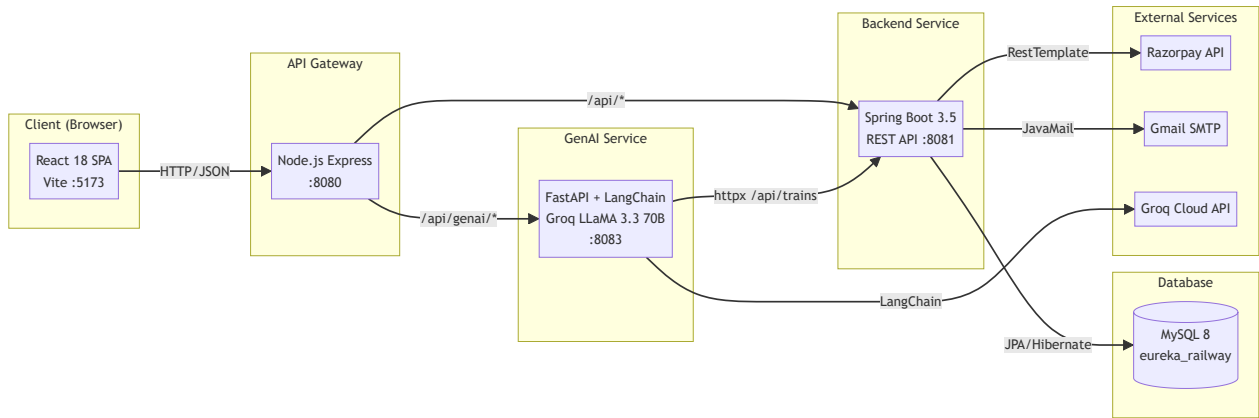


Fig 6.1 — High-Level System Architecture

6.2 Technology Stack Summary

Layer	Technology	Version	Responsibility
Presentation	React 18, React Router, Redux, Axios, React Icons, React Toastify	18.3.1	UI rendering, routing, state management, API calls
API Gateway	Node.js, Express, http-proxy-middleware, CORS	Express 5.2	Request routing, CORS, path rewriting
Business Logic	Spring Boot, Spring Web, Spring Data JPA, Spring Security, Spring AOP, Spring Mail, Lombok	3.5.4	REST API, business rules, security, logging
AI/ML	FastAPI, LangChain, ChatGroq, httpx, Pydantic	Python 3.10+	LLM-powered chatbot with live data context
Persistence	MySQL, Hibernate ORM	8.0/9.0	Relational data storage
Security	JWT (jjwt 0.11.5), BCrypt, HMAC-SHA256	—	Authentication, password hashing, payment verification
Payment	Razorpay Java SDK (RestTemplate)	Test Mode	Order creation, payment capture, refunds
Email	Spring Mail, Gmail SMTP	—	Booking & cancellation notifications

6.3 Backend Layered Architecture

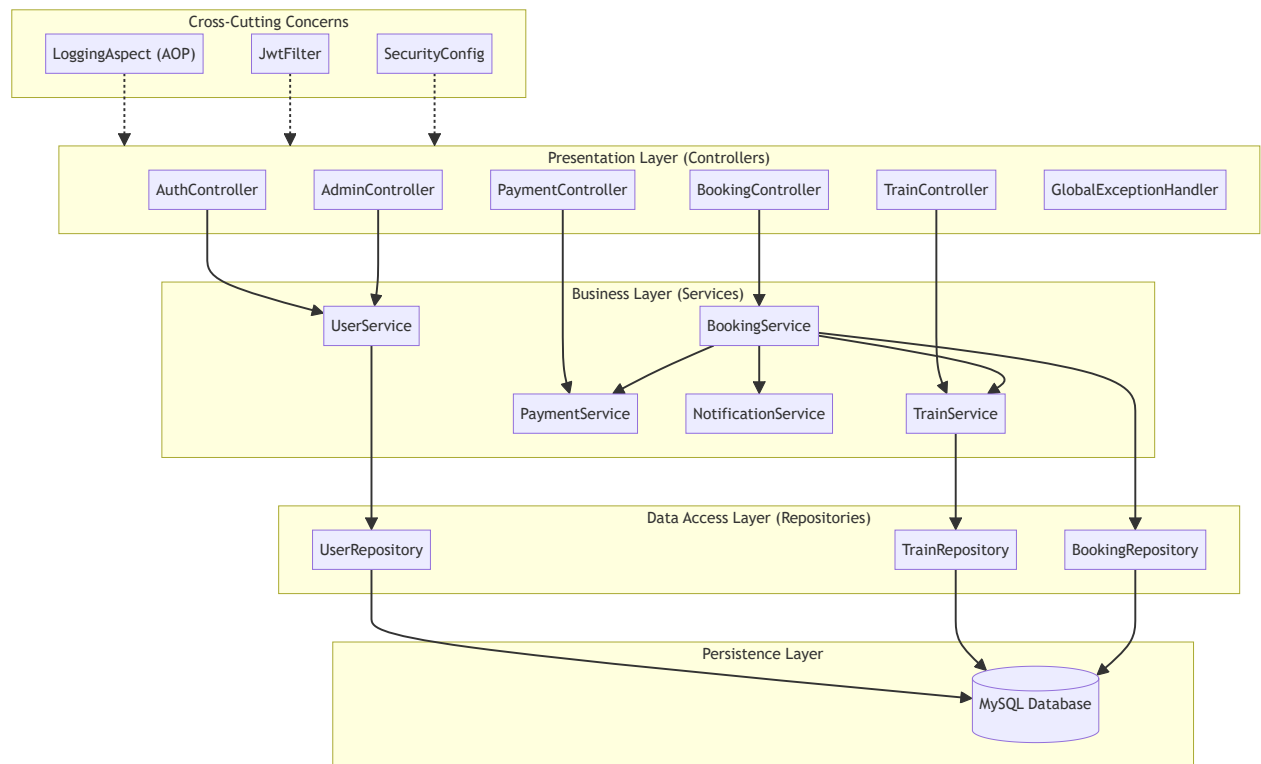


Fig 6.2 — Spring Boot Backend Layered Architecture

6.4 Request Processing Flow

HTTP Request → **JwtFilter** (validates token, sets SecurityContext) → **SecurityConfig** (checks role-based access rules) → **Controller** (receives JSON, delegates to service) → **Service** (applies business logic) → **Repository** (Spring Data JPA generates SQL) → **MySQL** (executes query) → **JSON Response**

Chapter 7: UML Diagrams

7.1 Use Case Diagram

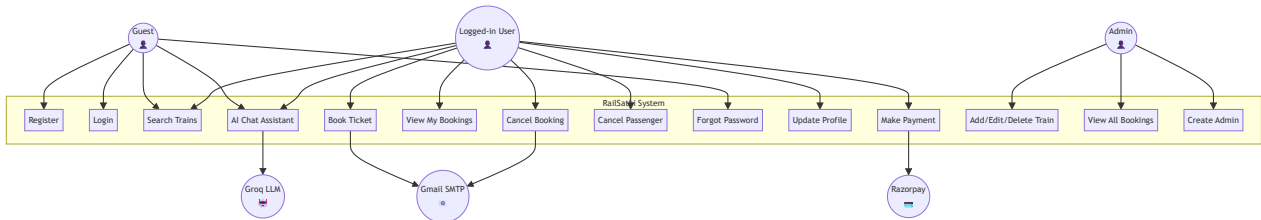
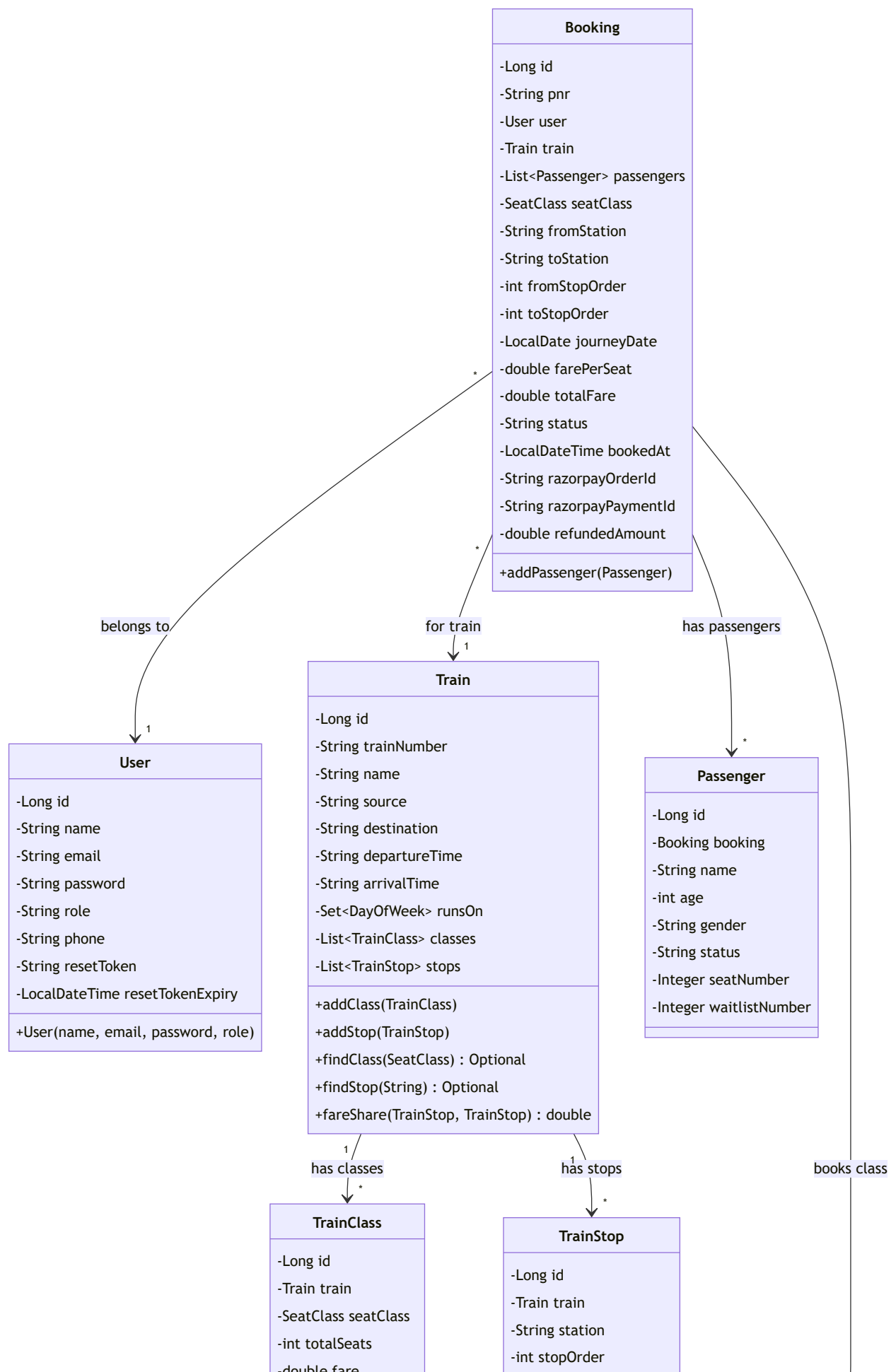


Fig 7.1 — Use Case Diagram

7.2 Class Diagram (Entity Layer)



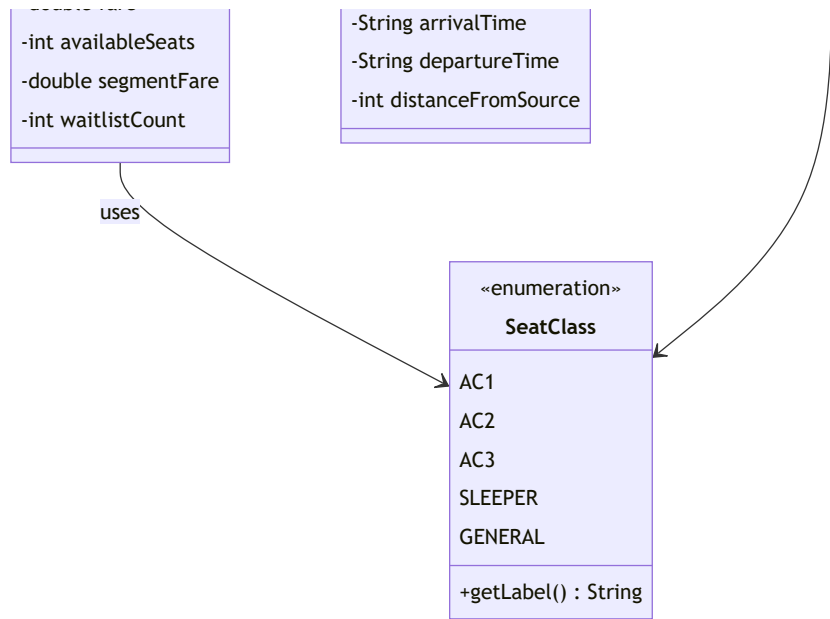


Fig 7.2 — Class Diagram (JPA Entities)

7.3 Sequence Diagrams

7.3.1 User Login Sequence

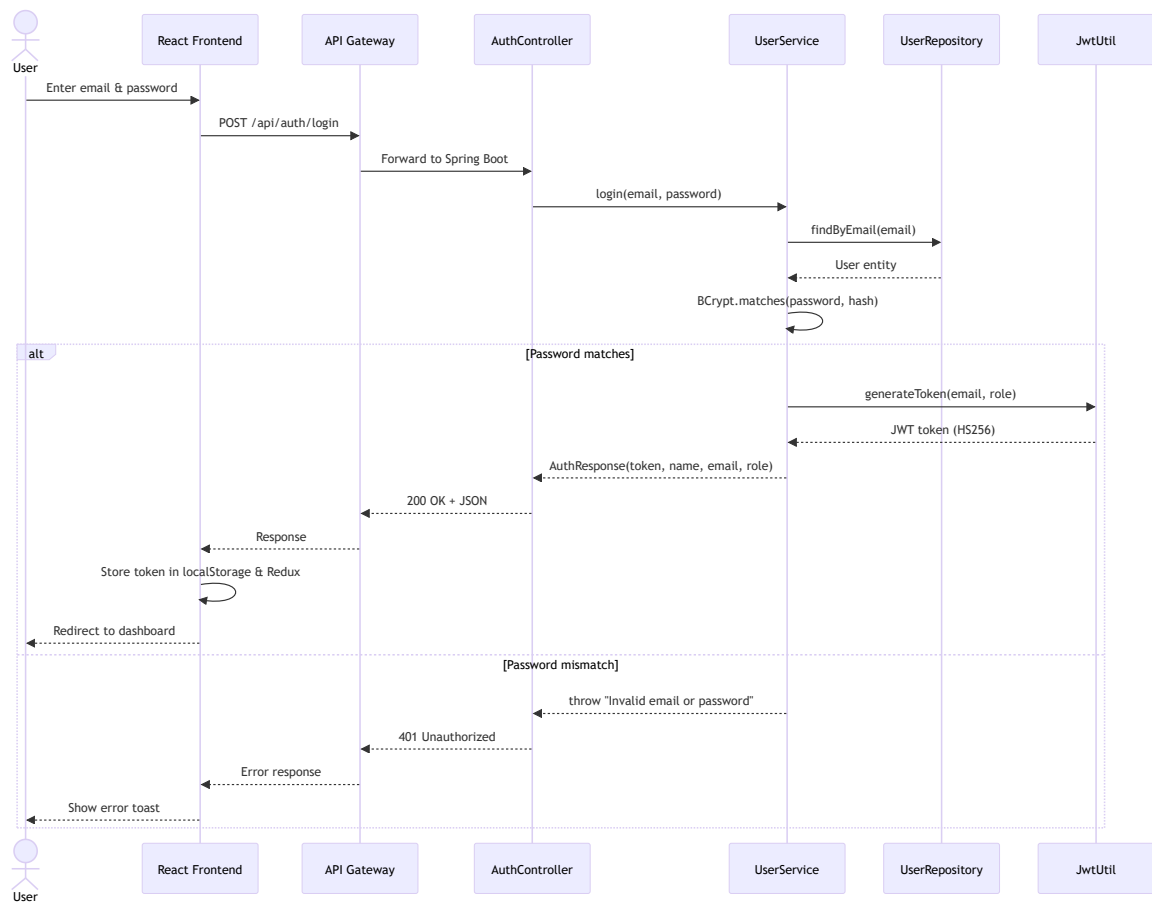


Fig 7.3.1 — Login Sequence Diagram

7.3.2 Booking with Payment Sequence

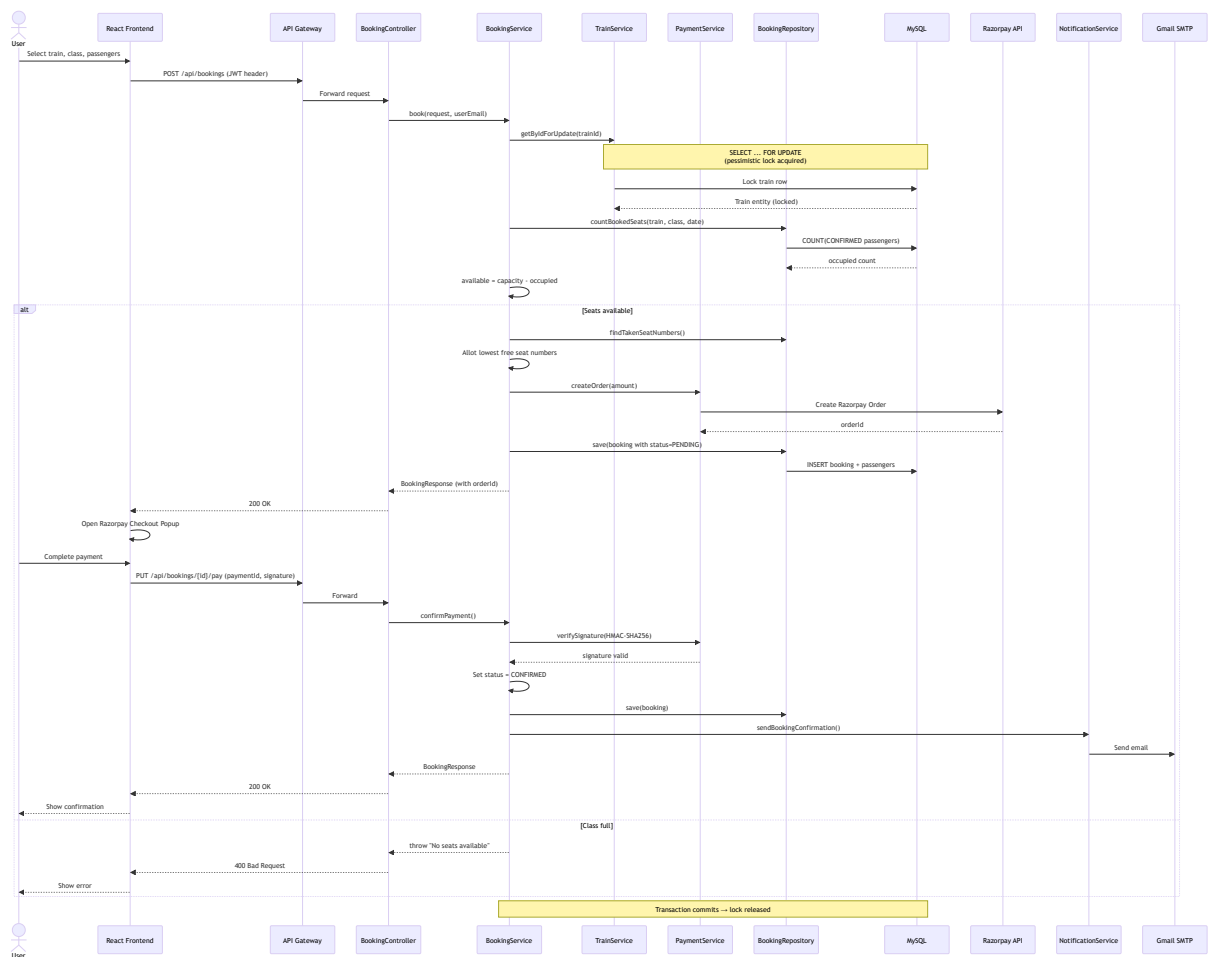


Fig 7.3.2 — Booking with Payment Sequence Diagram

7.3.3 Cancellation with Waitlist Promotion Sequence

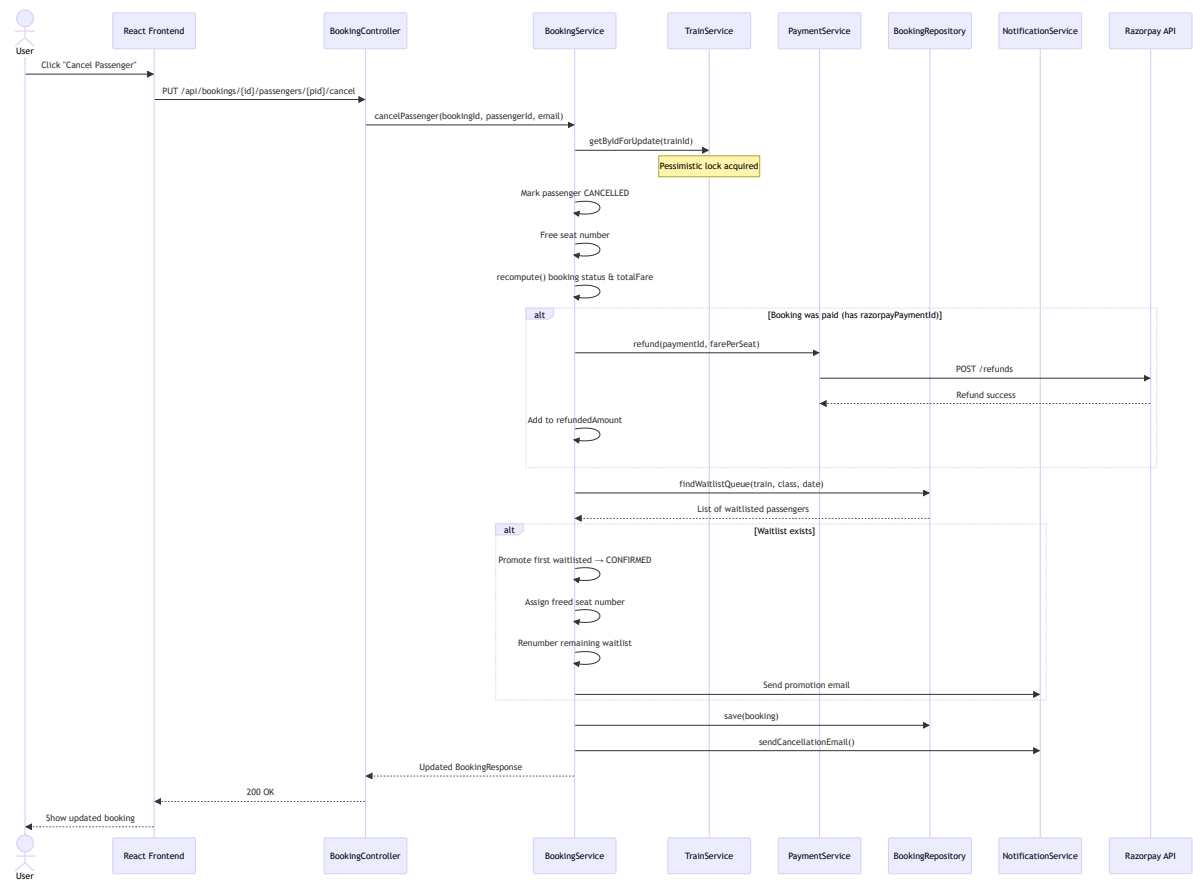
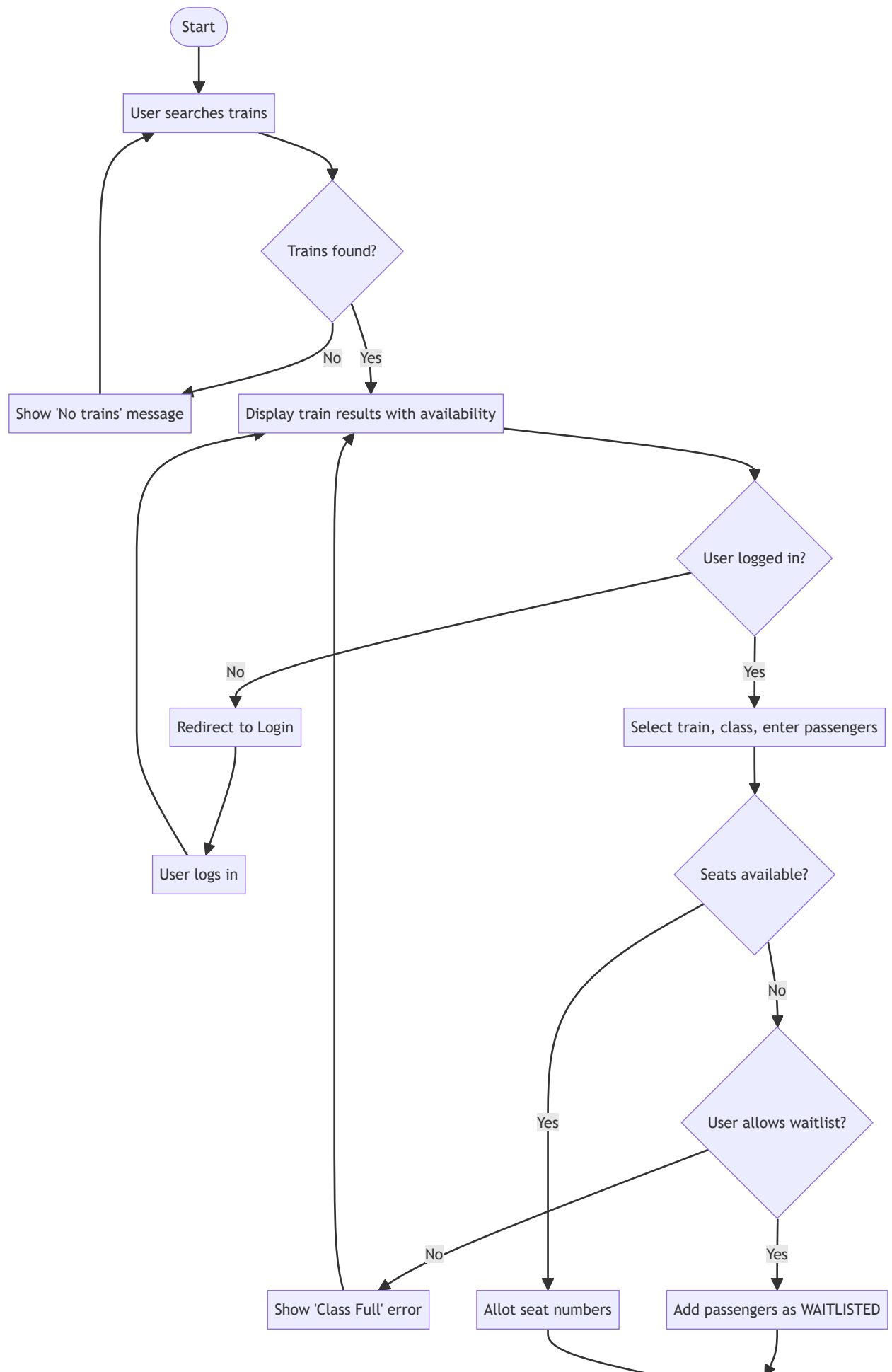


Fig 7.3.3 — Cancellation with Waitlist Promotion

7.4 Activity Diagram — Complete Booking Flow



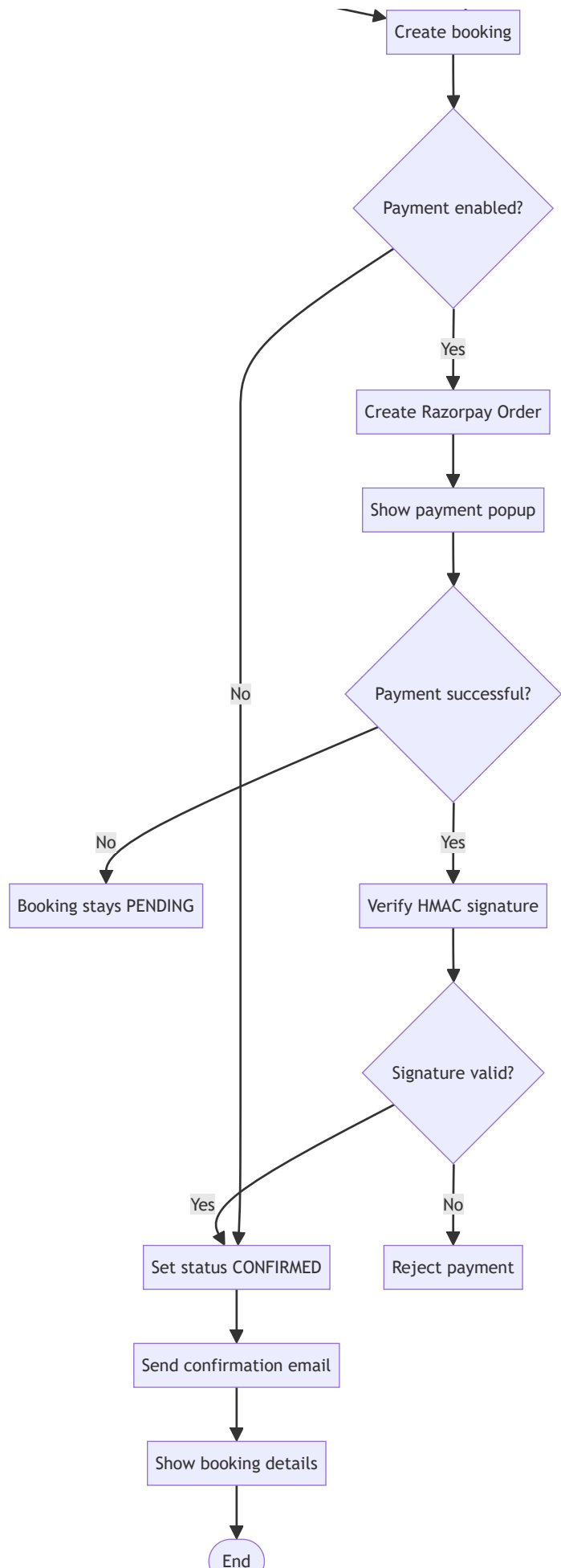




Fig 7.4 — Activity Diagram: Booking Flow

7.5 State Diagram — Booking Lifecycle

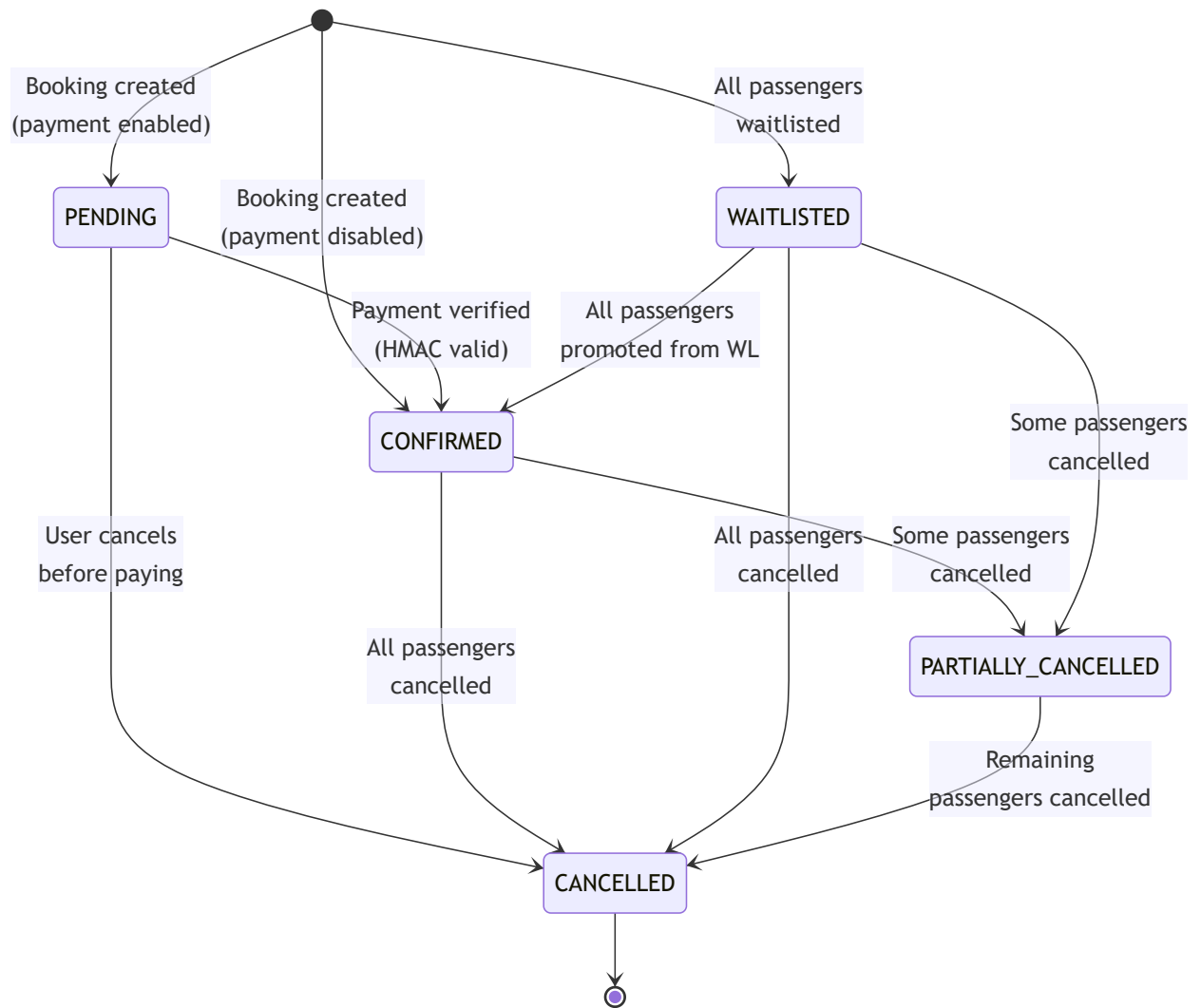


Fig 7.5 — State Diagram: Booking Status Lifecycle

7.6 State Diagram — Passenger Lifecycle

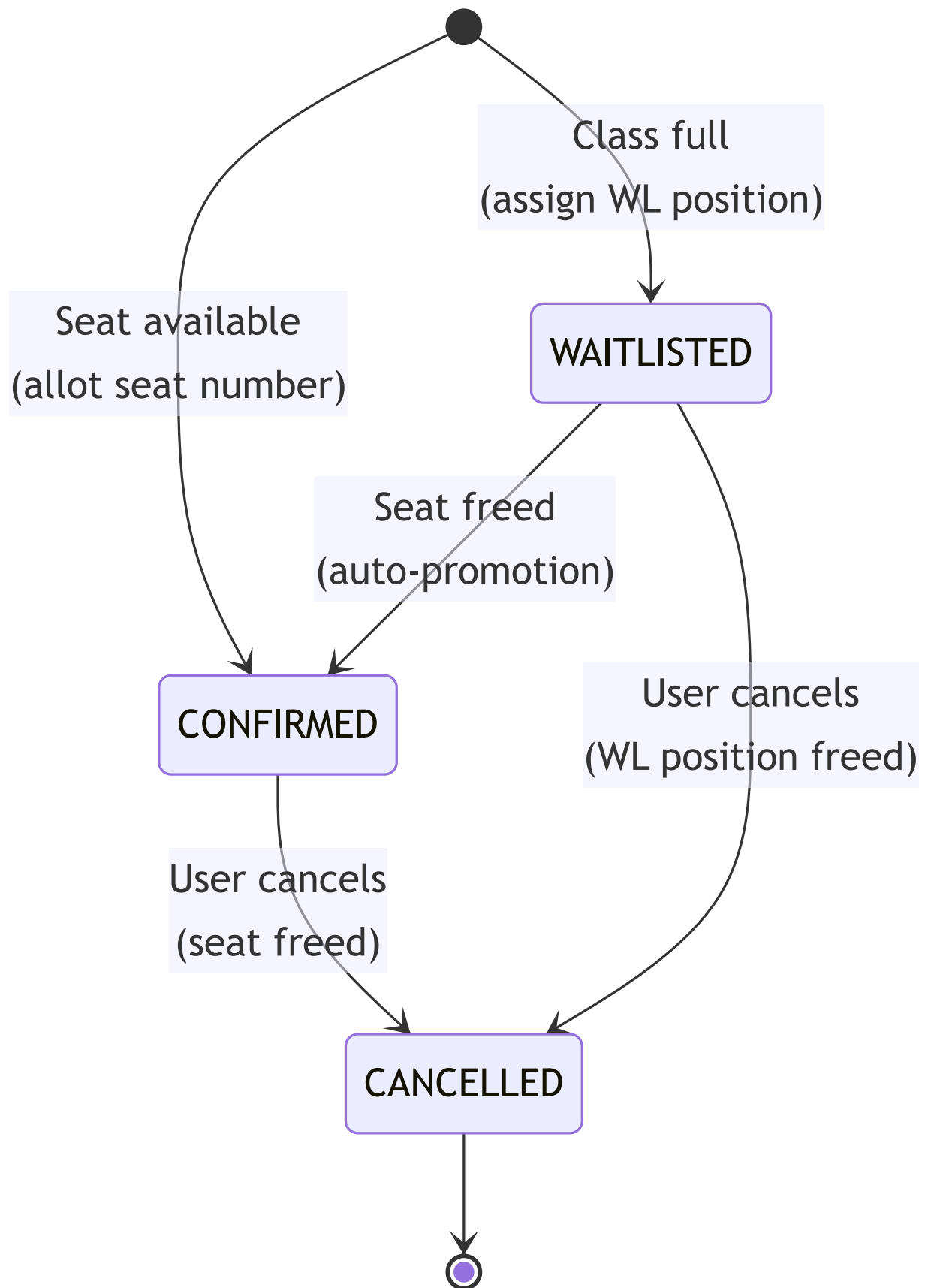


Fig 7.6 — State Diagram: Passenger Status Lifecycle

Chapter 8: Database Design

8.1 Entity-Relationship Diagram

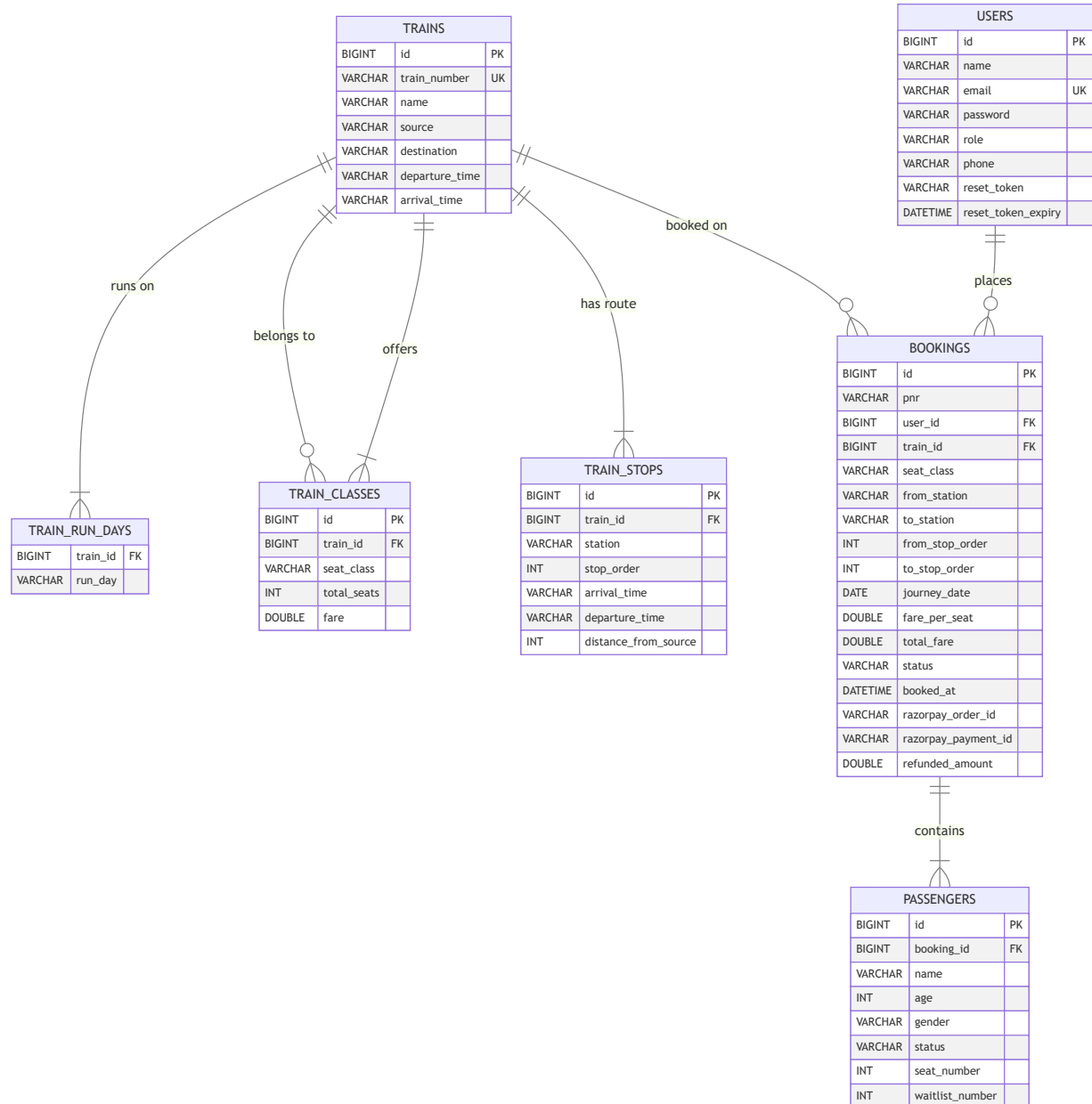


Fig 8.1 — Entity-Relationship Diagram

8.2 Table Descriptions

Table	Purpose	Key Columns	Relationships
users	Stores registered users and admins	id (PK), email (UK), password (BCrypt hash), role (USER/ADMIN)	1:N with bookings

trains	Master train information	id (PK), train_number (UK), source, destination, timings	1:N with train_classes, train_stops, train_run_days, bookings
train_classes	Travel classes offered per train	id (PK), train_id (FK), seat_class, total_seats, fare	N:1 with trains. UK on (train_id, seat_class)
train_stops	Route stops in order with distances	id (PK), train_id (FK), station, stop_order, distance_from_source	N:1 with trains. UK on (train_id, stop_order)
train_run_days	Days of week a train operates	train_id (FK), run_day (MONDAY–SUNDAY)	N:1 with trains (ElementCollection)
bookings	Ticket bookings with payment info	id (PK), pnr, user_id (FK), train_id (FK), seat_class, journey_date, status	N:1 with users and trains. 1:N with passengers
passengers	Individual passengers per booking	id (PK), booking_id (FK), name, age, gender, status, seat_number	N:1 with bookings

8.3 Key Database Queries (JPQL)

Query	Purpose	Used In
<code>SELECT COUNT(p) FROM Passenger p WHERE ... AND p.status = 'CONFIRMED'</code>	Count occupied seats per train/class/date	BookingRepository.countBookedSeats()
<code>SELECT p.seatNumber FROM Passenger p WHERE ... AND p.status = 'CONFIRMED'</code>	Find taken seat numbers for allotment	BookingRepository.findTakenSeatNumbers()
<code>SELECT t FROM Train t WHERE ... AND :day MEMBER OF t.runsOn</code>	Search trains by endpoints + day of week	TrainRepository.findByEndpointsOnDay()
<code>SELECT DISTINCT t FROM Train t JOIN t.stops boarding JOIN t.stops dropping WHERE ... AND boarding.stopOrder < dropping.stopOrder</code>	Intermediate station search (direction-aware)	TrainRepository.searchByStopsOnDay()
<code>SELECT t FROM Train t WHERE t.id = :id [PESSIMISTIC_WRITE]</code>	Acquire row lock for concurrent booking safety	TrainRepository.findByIdForUpdate()

Chapter 9: Module Description

9.1 Backend Modules

Module 1: Authentication & User Management

Aspect	Details
Files	AuthController, UserService, UserRepository, JwtUtil, JwtFilter, SecurityConfig
Functionality	Registration, login (JWT issuance), forgot/reset password, profile update, change password
Security	BCrypt password hashing, HS256 JWT signing, role-based access (ROLE_USER, ROLE_ADMIN)
Key Design	Stateless auth — no HTTP sessions. Token carries email + role. JwtFilter validates on every request.

Module 2: Train Management

Aspect	Details
Files	TrainController, TrainService, TrainRepository, Train, TrainClass, TrainStop, SeatClass, DataSeeder
Functionality	CRUD operations on trains (admin only), search by source/destination/date, intermediate station routing
Key Design	Separate TrainClass entity for per-class capacity/fare. TrainStop with stopOrder and distance for route modelling. Day-of-week filtering via @ElementCollection.

Module 3: Booking Engine

Aspect	Details
Files	BookingController, BookingService, BookingRepository, Booking, Passenger, Dtos.java
Functionality	Ticket booking (1-6 passengers), seat allotment, waitlist management, full/partial cancellation, PNR generation
Key Design	Pessimistic locking prevents overselling. Fare locked at booking time. Status derived from passengers. Available seats computed (not stored).

Module 4: Payment Integration

Aspect	Details
Files	PaymentController, PaymentService
Functionality	Razorpay order creation, HMAC-SHA256 signature verification, refund processing
Key Design	Server-side signature verification prevents frontend tampering. Payment toggleable via config. Refund before DB save ensures atomicity.

Module 5: Notification Service

Aspect	Details
Files	NotificationService
Functionality	Email notifications for booking confirmation, cancellation, waitlist promotion, password reset
Key Design	Toggled via app.mail.enabled config. Graceful fallback to console logging. Wrapped in try-catch to never break bookings.

Module 6: Cross-Cutting Concerns

Aspect	Details
Files	LoggingAspect, GlobalExceptionHandler
Functionality	AOP-based controller method logging with execution time. Centralized exception handling with meaningful HTTP responses.
Key Design	@Around advice on controller pointcut. Single aspect handles all cross-cutting logging.

9.2 Frontend Modules

Module	Files	Description
App Shell	App.jsx, main.jsx, Navbar.jsx, index.css	Route definitions, navigation bar with role-based links, global styling
Authentication	Login.jsx, Register.jsx, ForgotPassword.jsx, ResetPassword.jsx	Auth forms with Axios calls, token storage in localStorage & Redux
Train Search & Booking	Trains.jsx	Search form, results display with class availability, passenger entry modal, Razorpay checkout integration
Booking Management	MyBookings.jsx	View bookings (upcoming/past), cancel booking/passenger, pay pending bookings
User Profile	Profile.jsx	Edit name/phone, change password
Admin	AdminTrains.jsx, AllBookings.jsx, CreateAdmin.jsx	Train CRUD with classes and stops, view all system bookings, create admin accounts
AI Chatbot	Chatbot.jsx, Chatbot.css	Floating chat widget, message history, API integration with GenAI service
State Management	store.js	Redux store with authReducer (LOGIN, LOGOUT, UPDATE_USER actions)
API Layer	api.js, payment.js	Axios instance with JWT interceptor, Razorpay script loader

9.3 API Gateway Module

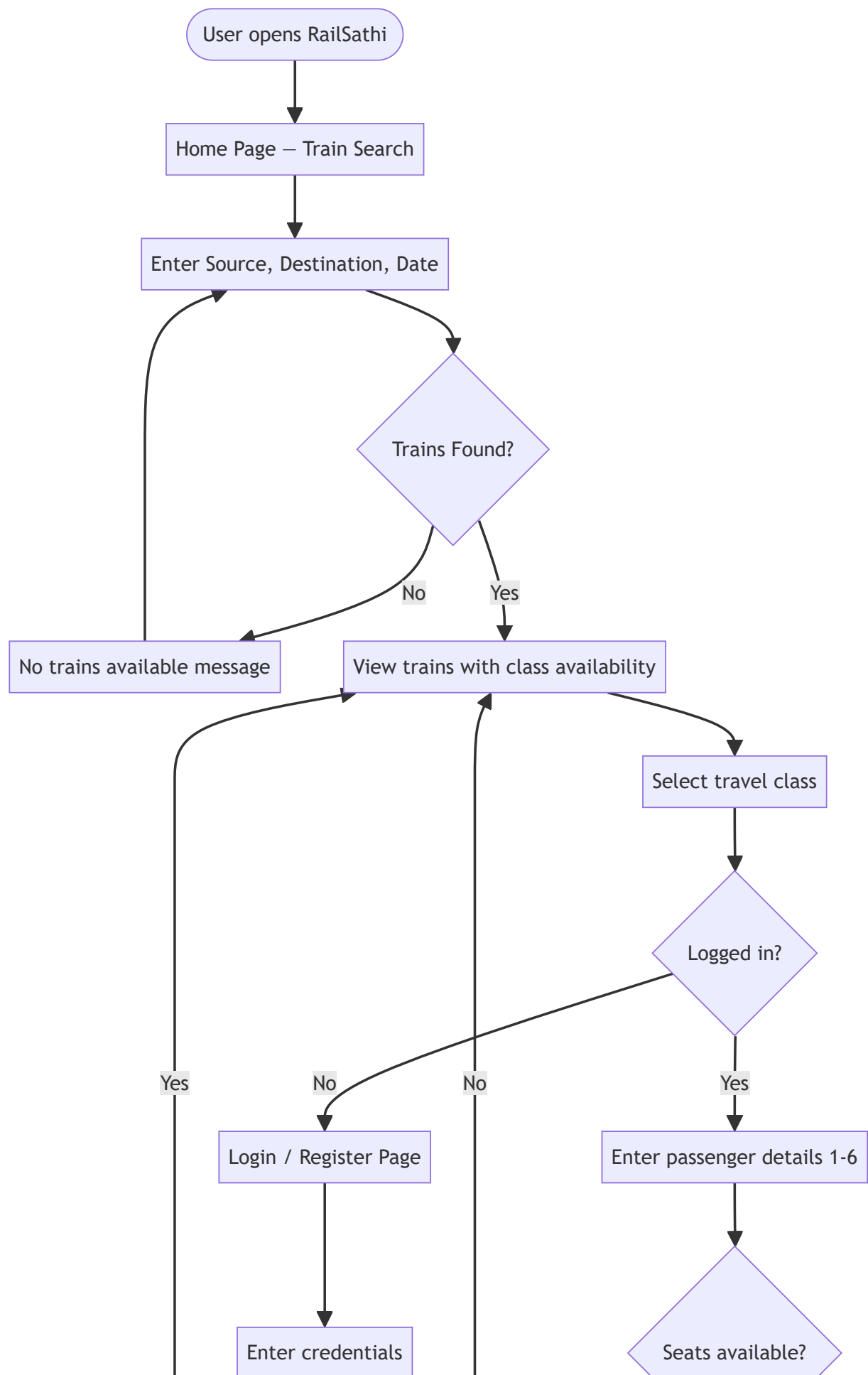
Aspect	Details
File	server.js
Technology	Express 5 + http-proxy-middleware
Routing	/api/genai/* → FastAPI (port 8083), /api/* → Spring Boot (port 8081)
Features	CORS configuration for frontend origin, path rewriting

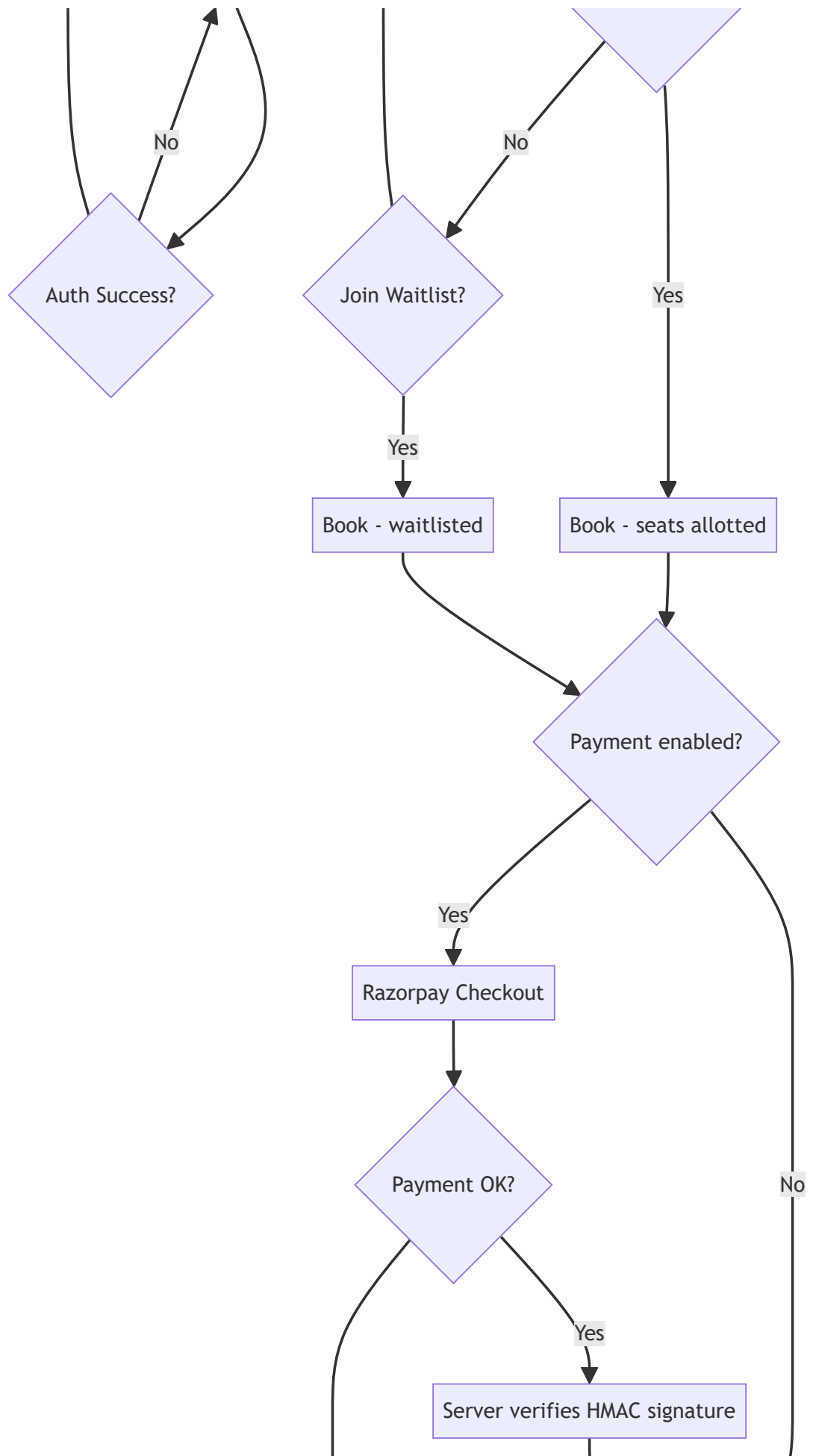
9.4 GenAI Service Module

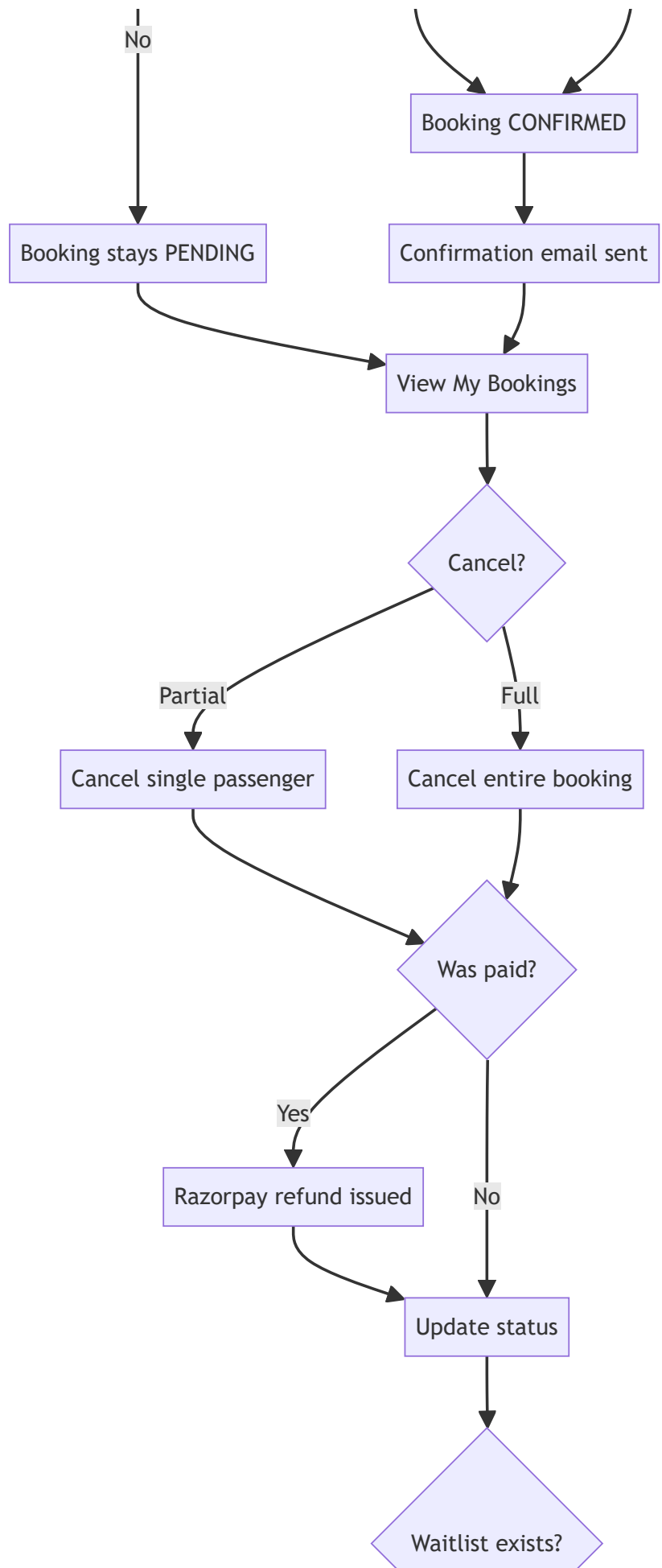
Aspect	Details
File	main.py
Technology	FastAPI + LangChain + ChatGroq
LLM	Groq Cloud — LLaMA 3.3 70B Versatile (temperature 0.3)
Flow	Receives user message → Fetches live train data from Spring Boot → Constructs system prompt with real data → Invokes LLM → Returns response

Chapter 10: Complete User Flow

10.1 Passenger (User) Flow







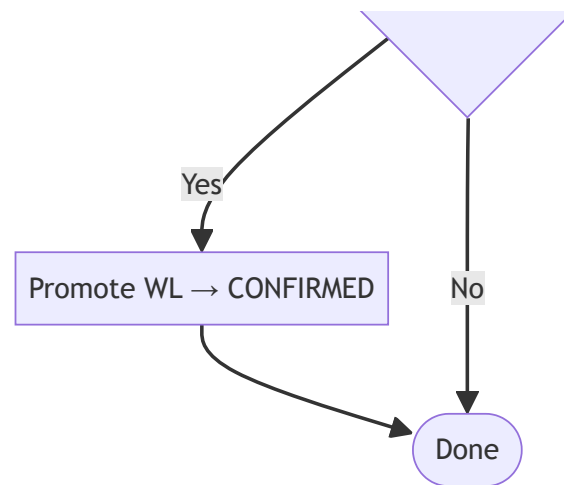


Fig 10.1 — Complete Passenger User Flow

10.2 Admin Flow

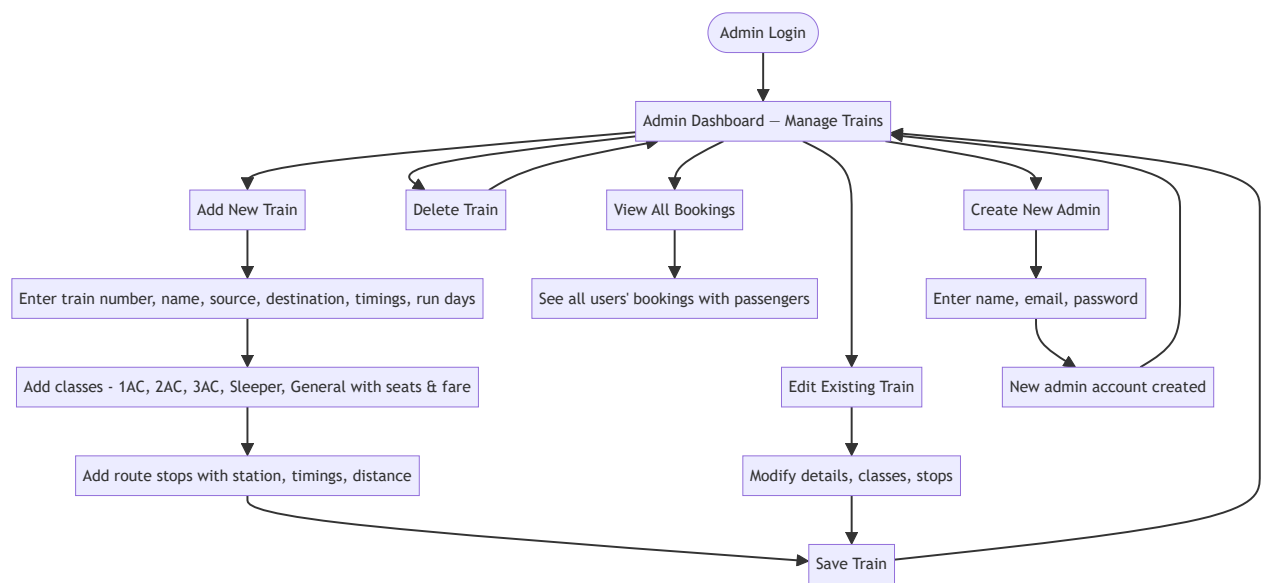


Fig 10.2 — Admin User Flow

10.3 AI Chatbot Flow

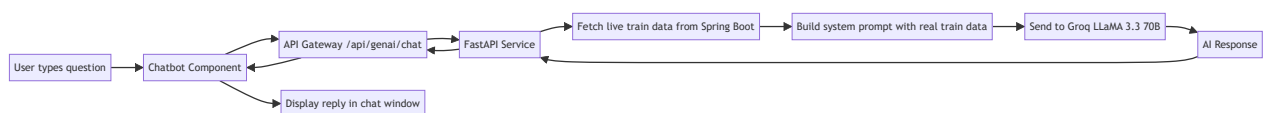


Fig 10.3 — AI Chatbot Data Flow

Chapter 11: REST API Specification

11.1 Authentication APIs

Method	Endpoint	Access	Description	Request Body
POST	/api/auth/register	Public	Create new user account	{name, email, password}
POST	/api/auth/login	Public	Authenticate and receive JWT	{email, password}
POST	/api/auth/forgot-password	Public	Send password reset link via email	{email}
POST	/api/auth/reset-password	Public	Set new password using reset token	{token, newPassword}
GET	/api/auth/me	Logged in	Get current user profile	—
PUT	/api/auth/profile	Logged in	Update name & phone	{name, phone}
PUT	/api/auth/password	Logged in	Change password	{currentPassword, newPassword}

11.2 Train APIs

Method	Endpoint	Access	Description
GET	/api/trains?source=&destination=&date=	Public	Search trains with availability
POST	/api/trains	ADMIN	Add new train with classes and stops
PUT	/api/trains/{id}	ADMIN	Update train details
DELETE	/api/trains/{id}	ADMIN	Delete train

11.3 Booking APIs

Method	Endpoint	Access	Description
POST	/api/bookings	Logged in	Book ticket with passenger list
GET	/api/bookings/my	Logged in	Get current user's bookings
GET	/api/bookings/all	ADMIN	Get all bookings in the system
PUT	/api/bookings/{id}/pay	Logged in	Verify Razorpay payment, confirm booking
PUT	/api/bookings/{id}/cancel	Logged in	Cancel entire booking (with refund)

PUT	/api/bookings/{id}/passengers/{pid}/cancel	Logged in	Cancel single passenger (partial cancellation)
-----	--	-----------	--

11.4 Payment & Admin APIs

Method	Endpoint	Access	Description
GET	/api/payments/config	Logged in	Payment enabled flag + Razorpay public key
POST	/api/admin/create-admin	ADMIN	Create a new admin account

11.5 GenAI API

Method	Endpoint	Access	Description
POST	/api/genai/chat	Public	Send message to AI chatbot, receive response

11.6 Sample Request / Response

Booking Request:

```
{
  "trainId": 1,
  "seatClass": "AC3",
  "journeyDate": "2026-08-15",
  "allowWaitlist": false,
  "fromStation": "Bhopal",
  "toStation": "Nagpur",
  "passengers": [
    { "name": "Amit", "age": 25, "gender": "Male" },
    { "name": "Neha", "age": 23, "gender": "Female" }
  ]
}
```

Authentication Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyQG...\"",
  "name": "Amit Kumar",
  "email": "amit@example.com",
  "role": "USER"
}
```

Chapter 12: Implementation Details

12.1 Concurrency Control — Pessimistic Locking

The most critical engineering challenge is preventing two users from booking the last available seat simultaneously. RailSathi solves this with a **pessimistic write lock**:

```
// TrainRepository.java
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT t FROM Train t WHERE t.id = :id")
Optional<Train> findByIdForUpdate(@Param("id") Long id);
```

At the start of `BookingService.book()`, the train row is locked with `SELECT ... FOR UPDATE`. Since the method is `@Transactional`, the lock is held until the transaction commits. A concurrent booking for the *same train* blocks on the lock, then re-counts seats and correctly sees the updated availability. Bookings for *different trains* lock different rows, so they proceed in parallel.

12.2 Seat Availability — Computed, Not Stored

```
available = classCapacity - COUNT(CONFIRMED passengers on train+class+date)
```

This approach ensures that cancellations (even partial) automatically free seats without maintaining a counter. The database is always the source of truth.

12.3 Part-Journey Fare Calculation

```
segmentFare = classFare * (toKm - fromKm) / (lastStopKm - firstStopKm)
```

The `fareShare()` method on the `Train` entity computes the proportional fare using the `distanceFromSource` values from the `TrainStop` entities. This is used both for displaying fares in search results and for locking the fare at booking time.

12.4 Payment Security — HMAC-SHA256 Verification

The Razorpay checkout runs entirely in the browser. To prevent a tampered frontend from faking a payment:

1. Backend creates a Razorpay *order* (server-to-server with the secret key)
2. Frontend opens the Razorpay popup, collects payment
3. Razorpay signs `orderId|paymentId` with the secret key (HMAC-SHA256)
4. Frontend sends `paymentId` + signature to `/api/bookings/{id}/pay`
5. Backend recomputes the signature with its copy of the secret key
6. Only if signatures match is the booking marked CONFIRMED

12.5 Waitlist Promotion Algorithm

1. When a seat is freed (cancellation), `promoteFromWaitlist()` is called

2. Fetch the waitlist queue ordered by waitlistNumber ASC
3. For each freed seat, promote the earliest waiting passenger:
 - Change status from WAITLISTED → CONFIRMED
 - Assign the freed seat number
 - Clear waitlistNumber
4. Renumber remaining waitlisted passengers (WL3 → WL2, etc.)
5. Send promotion email to each promoted passenger's booking user
6. Recompute the booking status of all affected bookings

12.6 Email Enumeration Prevention

The forgot-password endpoint returns the same message ("If that email is registered, a reset link has been sent") for both registered and unregistered emails. This prevents attackers from scripting address enumeration. The same principle applies to login ("Invalid email or password" — not distinguishing which is wrong).

12.7 DTO Pattern

Controllers return `BookingResponse` / `PassengerResponse` record DTOs instead of raw JPA entities. Benefits:

- API shape is explicit and decoupled from the database schema
- Prevents leaking nested entities (e.g., User inside Booking) into JSON
- Avoids lazy-loading / infinite recursion issues during serialization
- Java records are immutable and concise

12.8 Redux State Management

Redux is used *only* for authentication state (logged-in user), which is needed by multiple components (Navbar, route guards, booking page). Page-local data (train search results, form fields) uses React's `useState` — avoiding unnecessary Redux boilerplate.

Chapter 13: Testing

13.1 Testing Strategy

Test Type	Tool	Scope
Unit Testing	JUnit 5 + Mockito	BookingService business logic — seat allotment, fare calculation, cancellation
Concurrency Testing	JUnit 5 + H2 (in-memory DB)	Proves pessimistic lock prevents overselling (12 threads race for 5 seats)
API Testing	Postman	Manual endpoint validation for all REST APIs
Integration Testing	Spring Boot Test	End-to-end with embedded H2 database
UI Testing	Manual / Browser DevTools	All user flows, responsive layout, error handling

13.2 Unit Test Cases

Sr.	Test Case	Expected Result	Status
1	Register with valid details	User created, 200 OK	✓ Pass
2	Register with duplicate email	400 Bad Request, "Email is already registered"	✓ Pass
3	Login with correct credentials	JWT token returned	✓ Pass
4	Login with wrong password	401 Unauthorized	✓ Pass
5	Search trains by endpoints	Matching trains with availability	✓ Pass
6	Search by intermediate stations	Only trains with correct stop order returned	✓ Pass
7	Book ticket — seats available	Booking CONFIRMED, seats allotted	✓ Pass
8	Book ticket — class full, no waitlist	400 "No seats available"	✓ Pass
9	Book ticket — class full, waitlist allowed	Booking WAITLISTED, WL numbers assigned	✓ Pass
10	Cancel single passenger	Seat freed, status PARTIALLY_CANCELLED	✓ Pass
11	Cancel all passengers	Status CANCELLED, refund issued	✓ Pass
12	Cancellation promotes waitlisted passenger	WL1 becomes CONFIRMED with seat number	✓ Pass
13	Concurrent booking — 12 threads, 5 seats	Exactly 5 succeed, 7 fail (no overselling)	✓ Pass
14	Part-journey fare calculation	Fare proportional to distance travelled	✓ Pass

15	Admin creates another admin	New admin account created	✓ Pass
16	Non-admin accesses admin endpoint	403 Forbidden	✓ Pass
17	Forgot password — registered email	Reset token generated, email sent	✓ Pass
18	Forgot password — unknown email	Same success message (prevents enumeration)	✓ Pass
19	Reset password with expired token	400 "Token expired"	✓ Pass
20	AI chatbot responds with train data	Relevant answer with real train information	✓ Pass

13.3 Running Tests

```
cd backend
mvn test
```

Runs BookingServiceTest (Mockito unit tests) plus BookingConcurrencyTest (multi-threaded integration test with in-memory H2 database). All tests pass successfully.

Chapter 14: Future Enhancements

Sr.	Enhancement	Description
1	Segment-based seat availability	Currently seats are held for the entire train run. Enhancement: make availability an interval-overlap problem, allowing the same seat to be reused on disjoint segments (e.g., seat freed at Nagpur can be rebooked Nagpur→Bengaluru).
2	Tatkal booking	Implement premium quota booking with higher fares and time-window restrictions, similar to IRCTC's Tatkal scheme.
3	PNR Status Check	Public endpoint to check booking status by PNR number without logging in.
4	Live train tracking	Integration with real-time train position APIs for live location tracking on a map.
5	Mobile responsive PWA	Convert the React frontend into a Progressive Web App with offline support and push notifications.
6	Containerization	Dockerize all four services with Docker Compose for one-command deployment. Add CI/CD with Jenkins/GitHub Actions.
7	RAG-based chatbot	Enhance the AI chatbot with Retrieval-Augmented Generation (RAG) for querying booking-specific and policy-related information.
8	Multi-language support	i18n support for Hindi, Marathi, and other regional languages.
9	Ticket PDF generation	Generate downloadable PDF e-tickets with QR codes for each confirmed booking.
10	Analytics Dashboard	Admin analytics with booking trends, revenue reports, and occupancy charts.

Chapter 15: Conclusion

RailSathi successfully demonstrates the design and implementation of a production-grade, AI-enhanced railway reservation system that integrates all core technologies covered in the CDAC PG-CP AC curriculum.

Through this project, the team gained hands-on experience with:

- **Full-stack development** — building a decoupled React frontend communicating with a Spring Boot REST API through a Node.js API gateway
- **Relational database modelling** — designing a normalised schema with complex relationships (trains → classes → stops → bookings → passengers)
- **Security engineering** — implementing JWT-based stateless authentication, BCrypt password hashing, role-based access control, and HMAC payment verification
- **Concurrency control** — using pessimistic database locking to prevent data integrity issues in high-contention scenarios
- **Third-party integration** — integrating Razorpay for payments, Gmail SMTP for notifications, and Groq LLM for AI-powered chat
- **Software engineering practices** — clean layered architecture, DTO pattern, AOP cross-cutting concerns, comprehensive testing with Mockito and concurrency tests
- **AI/ML integration** — connecting a large language model to live application data through a context-aware prompt engineering approach

The system handles real-world complexity including intermediate-station routing, distance-proportional fare calculation, waitlist management with automatic promotion, partial cancellation with proportional refunds, and email enumeration prevention — going well beyond a basic CRUD application.

RailSathi serves as a strong foundation that can be extended with features like segment-based availability, Tatkal booking, live tracking, and containerised deployment to approach production readiness.

Chapter 16: References

1. Spring Boot Documentation — <https://docs.spring.io/spring-boot/>
2. Spring Data JPA Reference — <https://docs.spring.io/spring-data/jpa/>
3. Spring Security Reference — <https://docs.spring.io/spring-security/>
4. React Documentation — <https://react.dev/>
5. Redux Documentation — <https://redux.js.org/>
6. React Router Documentation — <https://reactrouter.com/>
7. Vite Documentation — <https://vite.dev/>
8. MySQL 8 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/>
9. Razorpay Documentation — <https://razorpay.com/docs/>
10. FastAPI Documentation — <https://fastapi.tiangolo.com/>
11. LangChain Documentation — <https://python.langchain.com/>
12. Groq API Documentation — <https://console.groq.com/docs/>
13. JSON Web Token (JWT) — RFC 7519 — <https://datatracker.ietf.org/doc/html/rfc7519>
14. jjwt Library — <https://github.com/jwt/jwt>
15. Lombok Project — <https://projectlombok.org/>
16. Express.js Documentation — <https://expressjs.com/>
17. http-proxy-middleware — <https://github.com/chimurai/http-proxy-middleware>
18. IRCTC — Indian Railway Catering and Tourism Corporation — <https://www.irctc.co.in/>

Chapter 17: Team Contribution

Sr.	Team Member	Primary Responsibility	Modules Contributed
1	Ayush Phapale	Database Design & Entity Layer	ER diagram design, JPA entities (User, Train, TrainClass, TrainStop, Booking, Passenger), repositories, DataSeeder, MySQL schema
2	Komal Chavan	Business Logic & Services	BookingService (seat allotment, waitlist, cancellation, fare calculation), TrainService (search, CRUD), unit tests (Mockito), concurrency test
3	Mitali Raut	Security, AOP & Exception Handling	JwtUtil, JwtFilter, SecurityConfig (role-based access), BCrypt integration, LoggingAspect (AOP), GlobalExceptionHandler, PaymentService (Razorpay), NotificationService (email)
4	Pratik Chavan	React Frontend — Auth & Booking	Login, Register, ForgotPassword, ResetPassword pages. Trains page (search + booking modal + Razorpay checkout). MyBookings page (view, cancel, pay). API Gateway (Express). GenAI Service (FastAPI + LangChain + Groq).
5	Pratik Dhopte	React Frontend — Admin, State & Styling	Redux store (authReducer), React Router (route guards), Navbar, AdminTrains page (train CRUD with classes/stops), AllBookings, CreateAdmin, Profile page, Chatbot component, index.css (complete application styling)

Note: While each member had a primary responsibility, all team members contributed to code reviews, testing, and debugging across all modules. Every member can explain the complete system end-to-end.

Appendix A: Project Directory Structure

```

TrainWithGenAi/
├── api-gateway/                                # Node.js Express API Gateway (port 8080)
│   ├── server.js                             # Proxy routing: /api/genai → FastAPI, /api → Spring
└── Boot
    ├── package.json                         # Express, http-proxy-middleware, cors, dotenv
    └── .env                                # FRONTEND_URL, SPRING_BOOT_URL, FASTAPI_URL
├── backend/                                # Spring Boot 3.5 REST API (port 8081)
│   └── pom.xml                             # Maven dependencies (JPA, Security, AOP, Mail, JWT,
Lombok)
│   └── src/main/java/com/eureka/railway/
│       ├── RailwayApplication.java          # Spring Boot entry point
│       ├── entity/
│       │   ├── User.java                   # users table (id, name, email, password, role)
│       │   └── Train.java                  # trains table (trainNumber, source, destination,
timings)
│       │   ├── TrainClass.java             # train_classes table (seatClass, totalSeats, fare)
│       │   ├── TrainStop.java             # train_stops table (station, stopOrder, distance)
│       │   └── Booking.java                # bookings table (pnr, journeyDate, fare, status,
razorpay)
│       │   ├── Passenger.java              # passengers table (name, age, gender, seatNumber,
status)
│       │   ├── SeatClass.java              # Enum: AC1, AC2, AC3, SLEEPER, GENERAL
│       │   └── repository/                 # Spring Data JPA Repositories
│       │       ├── UserRepository.java
│       │       ├── TrainRepository.java    # pessimistic lock, intermediate station search
│       │       └── BookingRepository.java  # custom JPQL: countBookedSeats,
findTakenSeatNumbers
│       ├── service/                        # Business Logic Layer
│       │   ├── UserService.java           # auth, profile, password reset
│       │   ├── TrainService.java          # CRUD, search with availability
│       │   ├── BookingService.java        # booking, cancellation, waitlist, seat allotment
│       │   ├── PaymentService.java        # Razorpay order/verify/refund
│       │   └── NotificationService.java    # email notifications (Gmail SMTP)
│       ├── controller/                    # REST Controllers
│       │   ├── AuthController.java
│       │   ├── TrainController.java
│       │   ├── BookingController.java
│       │   ├── PaymentController.java
│       │   ├── AdminController.java
│       │   └── GlobalExceptionHandler.java
│       ├── security/                      # JWT + Spring Security
│       │   ├── JwtUtil.java               # token generation & validation (HS256)
│       │   ├── JwtFilter.java             # per-request token filter
│       │   └── SecurityConfig.java         # filter chain, CORS, role-based rules
│       ├── aspect/
│       │   └── LoggingAspect.java          # AOP @Around advice on controllers
│       ├── config/
│       │   └── DataSeeder.java             # seeds admin + sample trains on first run
│       └── dto/
│           └── Dtos.java                   # request/response records
├── frontend/                              # React 18 SPA (Vite, port 5173)
│   ├── package.json                       # React, Redux, Router, Axios, Toastify, React Icons
│   ├── vite.config.js
│   └── src/

```

```
├─ App.jsx # Route definitions with role-based guards
├─ main.jsx # ReactDOM.createRoot + Provider + BrowserRouter
├─ store.js # Redux store (authReducer: LOGIN, LOGOUT, UPDATE_USER)
├─ api.js # Axios instance with JWT interceptor
├─ payment.js # Razorpay script loader
├─ index.css # Complete application styles
├─ pages/ # Page Components
│   ├── Login.jsx, Register.jsx, ForgotPassword.jsx, ResetPassword.jsx
│   ├── Trains.jsx # Search + booking + Razorpay checkout
│   ├── MyBookings.jsx # View/cancel bookings
│   ├── Profile.jsx # Edit profile, change password
│   ├── AdminTrains.jsx # Train CRUD (classes + stops)
│   ├── AllBookings.jsx # Admin: view all bookings
│   ├── CreateAdmin.jsx # Admin: create admin account
│   └─ About.jsx # About page
├─ components/
│   ├── Navbar.jsx # Role-based navigation bar
│   ├── Chatbot.jsx # Floating AI chat widget
│   └─ Chatbot.css
├─ genai-service/ # Python FastAPI GenAI Service (port 8083)
│   ├── main.py # /chat endpoint: fetch trains → LLM prompt → response
│   └─ .env # GROQ_API_KEY, SPRING_BOOT_URL
├─ README.md # Project overview and setup instructions
├─ PROJECT_NOTES.md # Internal notes, design decisions, viva preparation
└─ .gitignore
```

Appendix B: Component Interaction Matrix

Component	React Frontend	API Gateway	Spring Boot	GenAI Service	MySQL	Razorpay	Gmail SMTP	Groq API
React Frontend	—	HTTP/JSON	—	—	—	JS SDK	—	—
API Gateway	CORS	—	Proxy	Proxy	—	—	—	—
Spring Boot	—	—	—	—	JPA/JDBC	RestTemplate	JavaMail	—
GenAI Service	—	—	httpx	—	—	—	—	LangChain

Appendix C: Default Credentials

Role	Email	Password	Notes
Admin	admin@railway.com	admin123	Created by DataSeeder on first run
User	(register via UI)	—	Self-registration from the Register page

Appendix D: Running the Application

Service	Directory	Command	Port
MySQL	—	Ensure MySQL is running; DB created automatically	3306
Backend	backend/	mvn spring-boot:run	8081
GenAI Service	genai-service/	python main.py	8083
API Gateway	api-gateway/	npm start	8080
Frontend	frontend/	npm run dev	5173

Open <http://localhost:5173> in a browser to access the application.

— **End of Report** —

RailSathi — AI-Powered Railway Reservation System

CDAC PG-CP AC | 2025–2026
